8

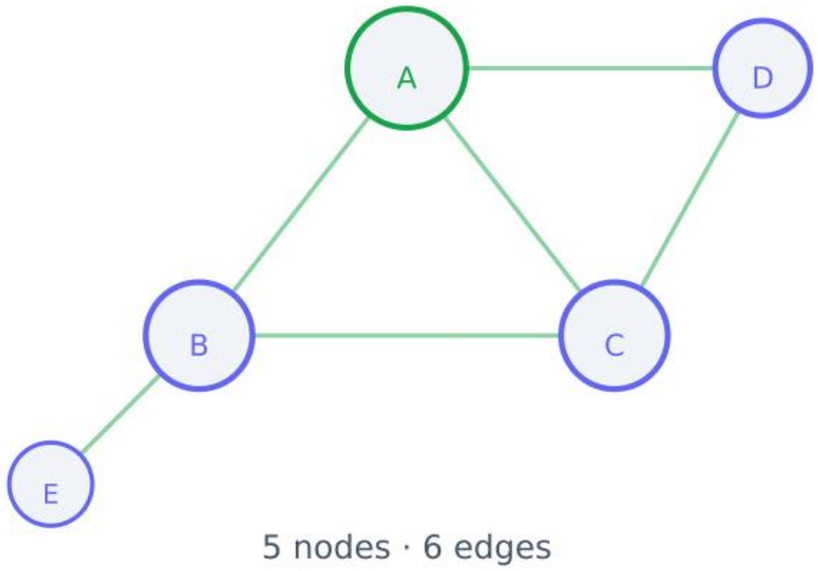
Graph Data Cleaning

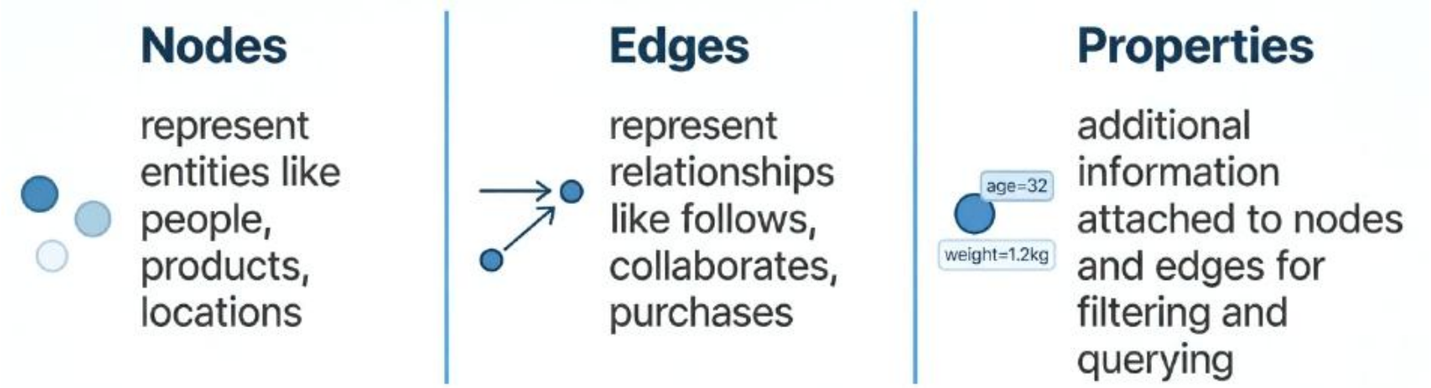
April, 2026

Rachid BENOUINI

What is a Graph?

- A graph is a mathematical structure for representing relationships. It consists of two key components:
- **Nodes (Vertices)**
  - The entities in your data: people, web pages, cities, proteins, accounts.
- **Edges (Links)**
  - The relationships between entities: friendships, hyperlinks, roads, reactions.
- Almost any dataset that involves relationships can be modelled as a graph.

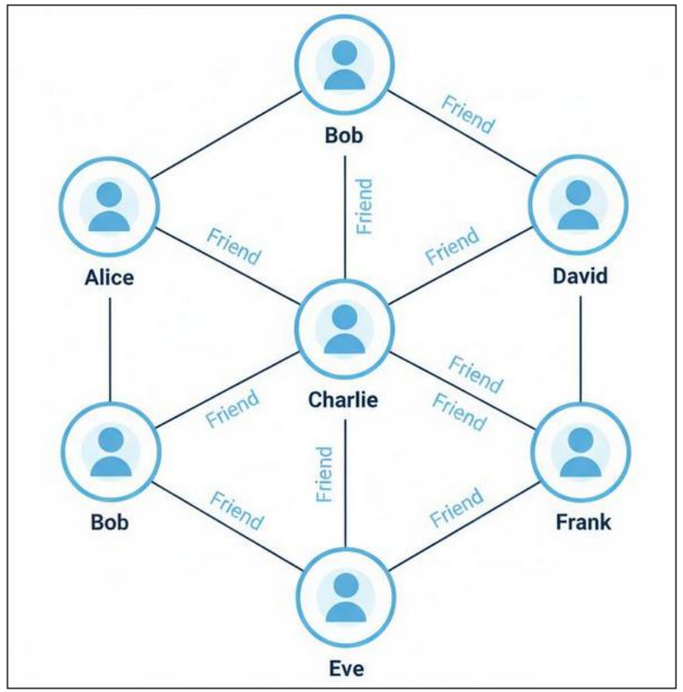




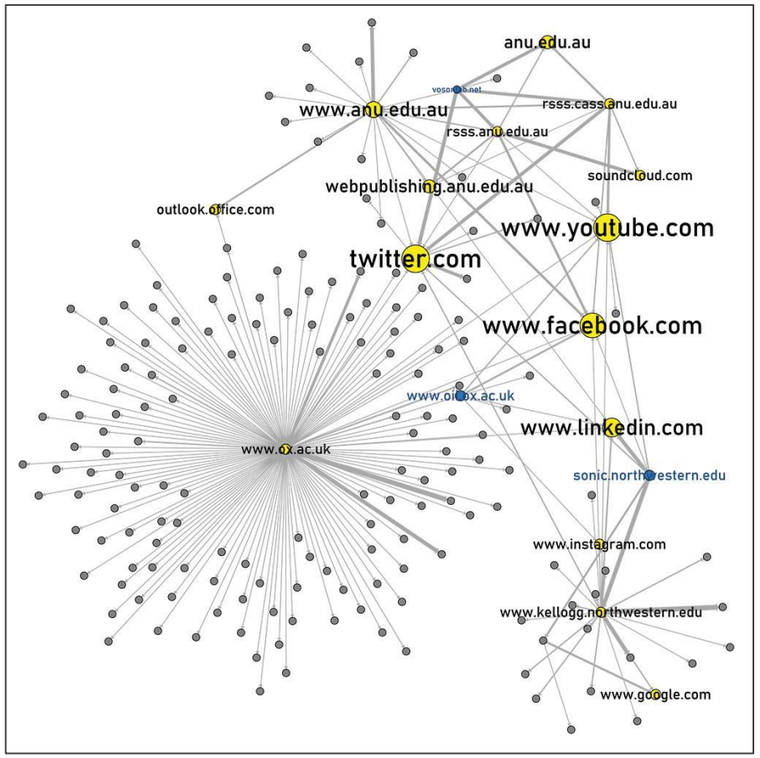
Graphs Are Everywhere

- Real-world systems that are naturally modelled as graphs:

Social networks
Users = nodes. Friendships or follows = edges.
Used by Facebook, LinkedIn, Twitter.



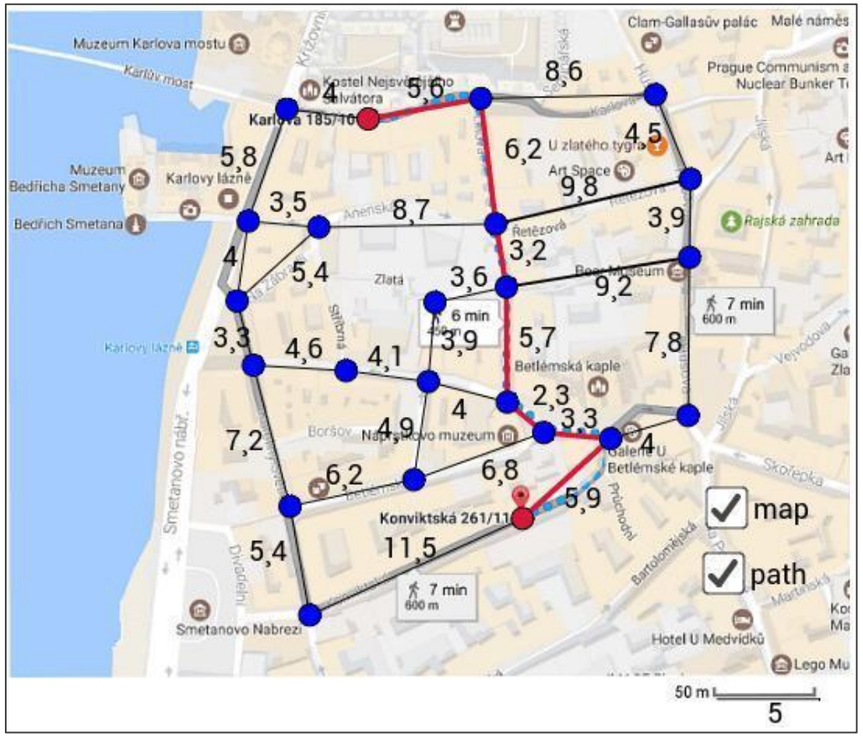
Web & knowledge graphs
Web pages = nodes. Hyperlinks = edges. Google's PageRank is a graph algorithm.



Graphs Are Everywhere

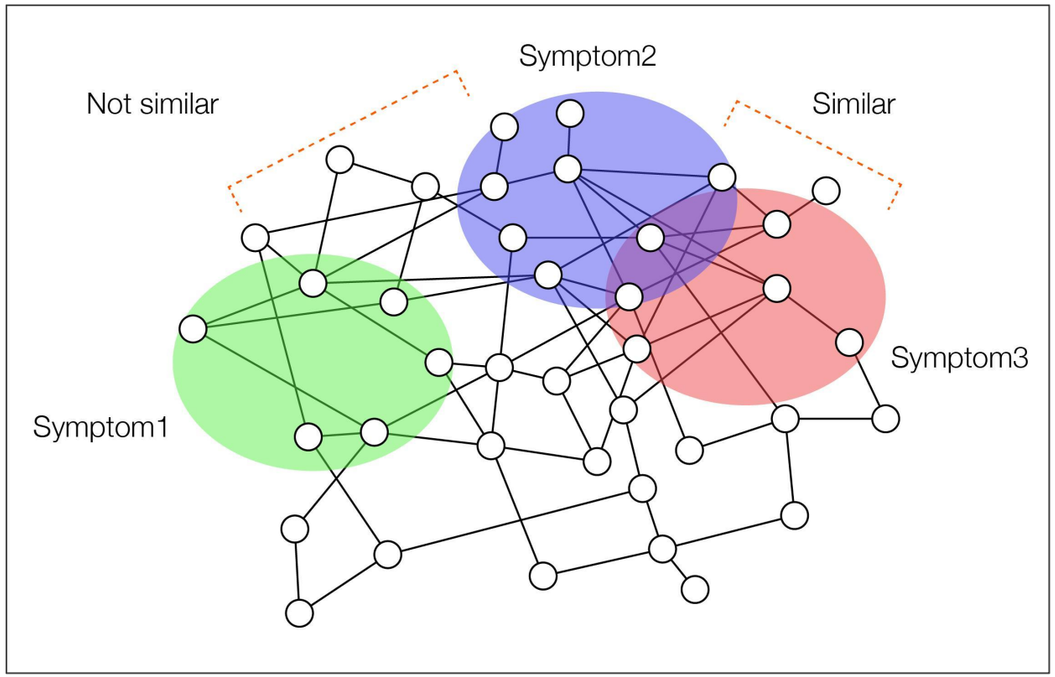
Transport networks

Cities = nodes. Roads/rails = edges. GPS routing solves shortest-path on a graph.



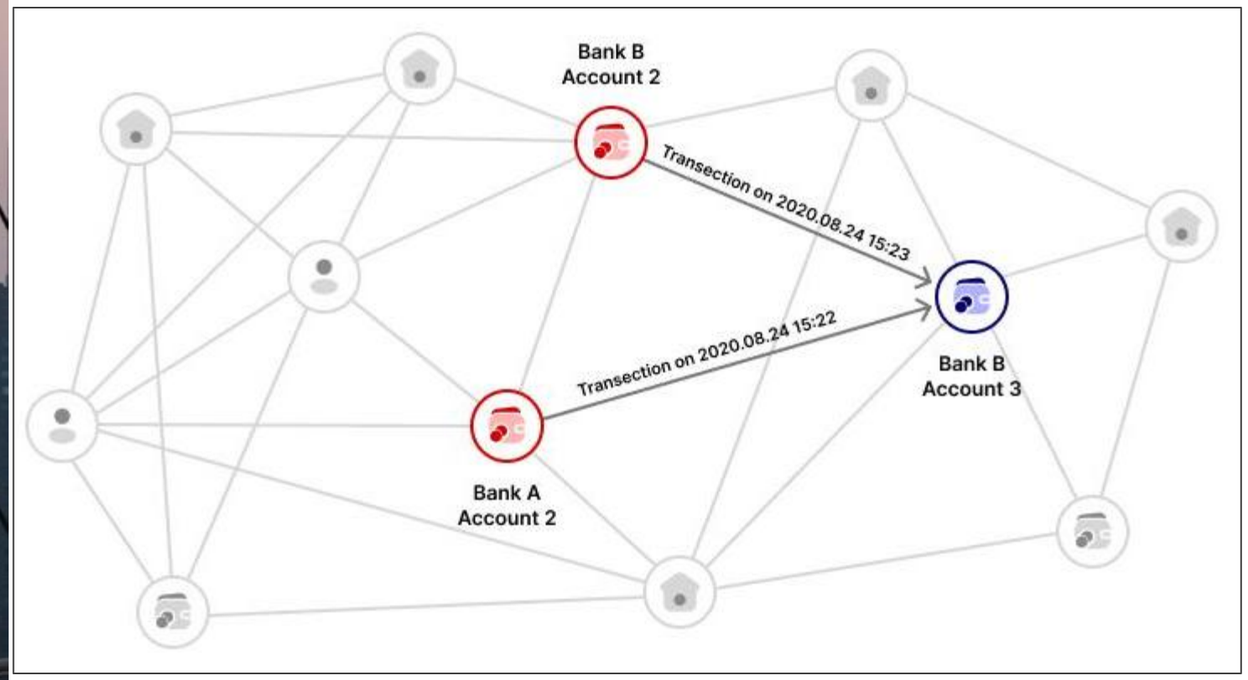
Biological networks

Proteins = nodes. Interactions = edges. Essential for drug discovery research.

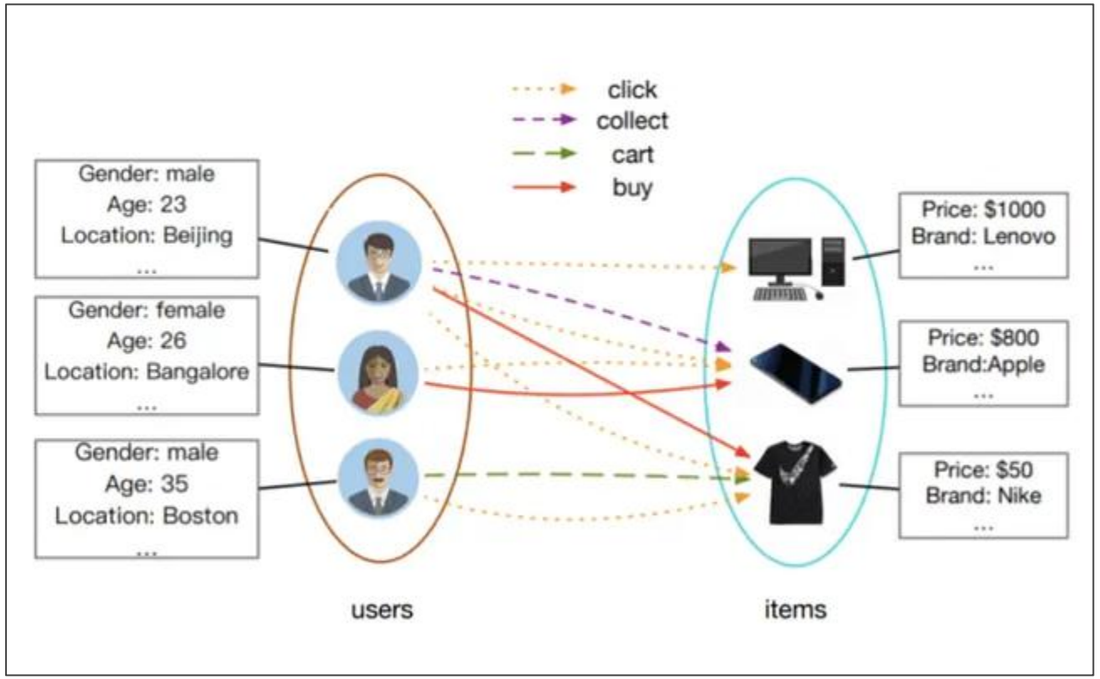


Graphs Are Everywhere

Fraud detection
Accounts = nodes. Transactions = edges.
Suspicious clusters reveal fraud rings.



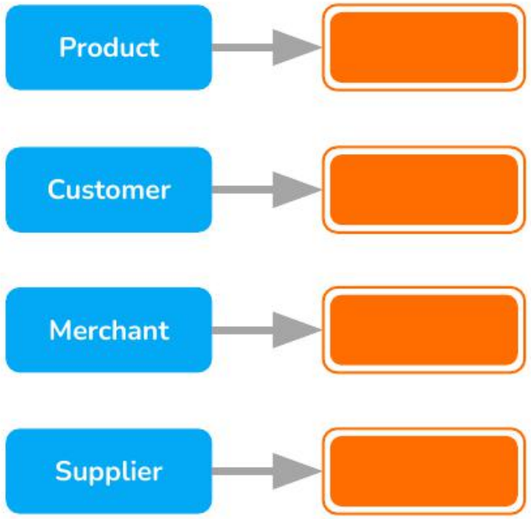
Recommendation engines
Users & products = nodes. Purchases = edges.
Amazon & Netflix use graph models.



Graph Stores Data Differently

Key value

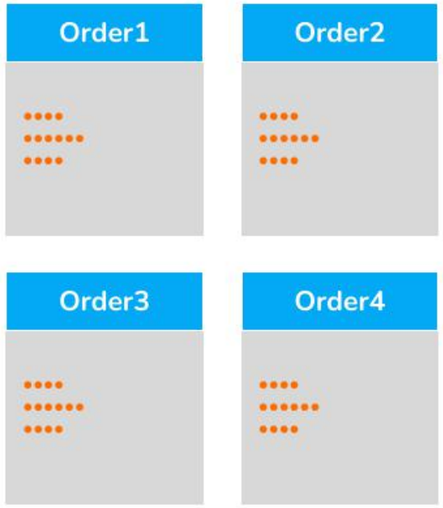
High speed storage / retrieval apps



Shopping carts, real-time bidding, messaging, and chat

Document

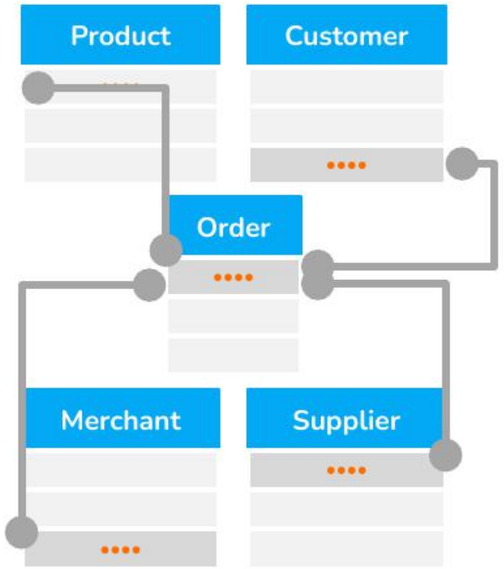
Front-end apps with unstructured data



Gaming, content management, mobile apps, and IoT

Relational

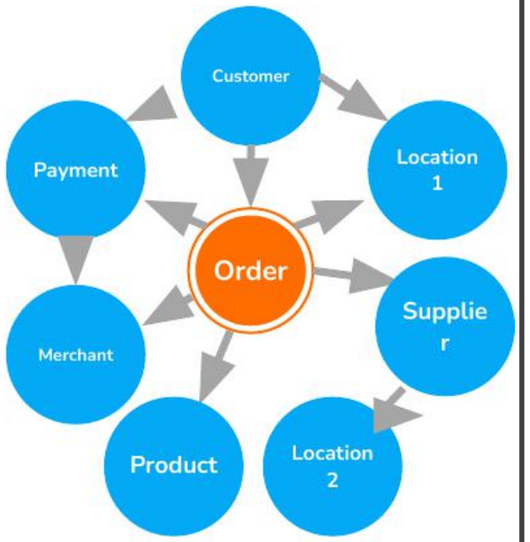
Transactional systems of record with structured data



Customer billing, payroll, inventory monitoring, and bank accounts

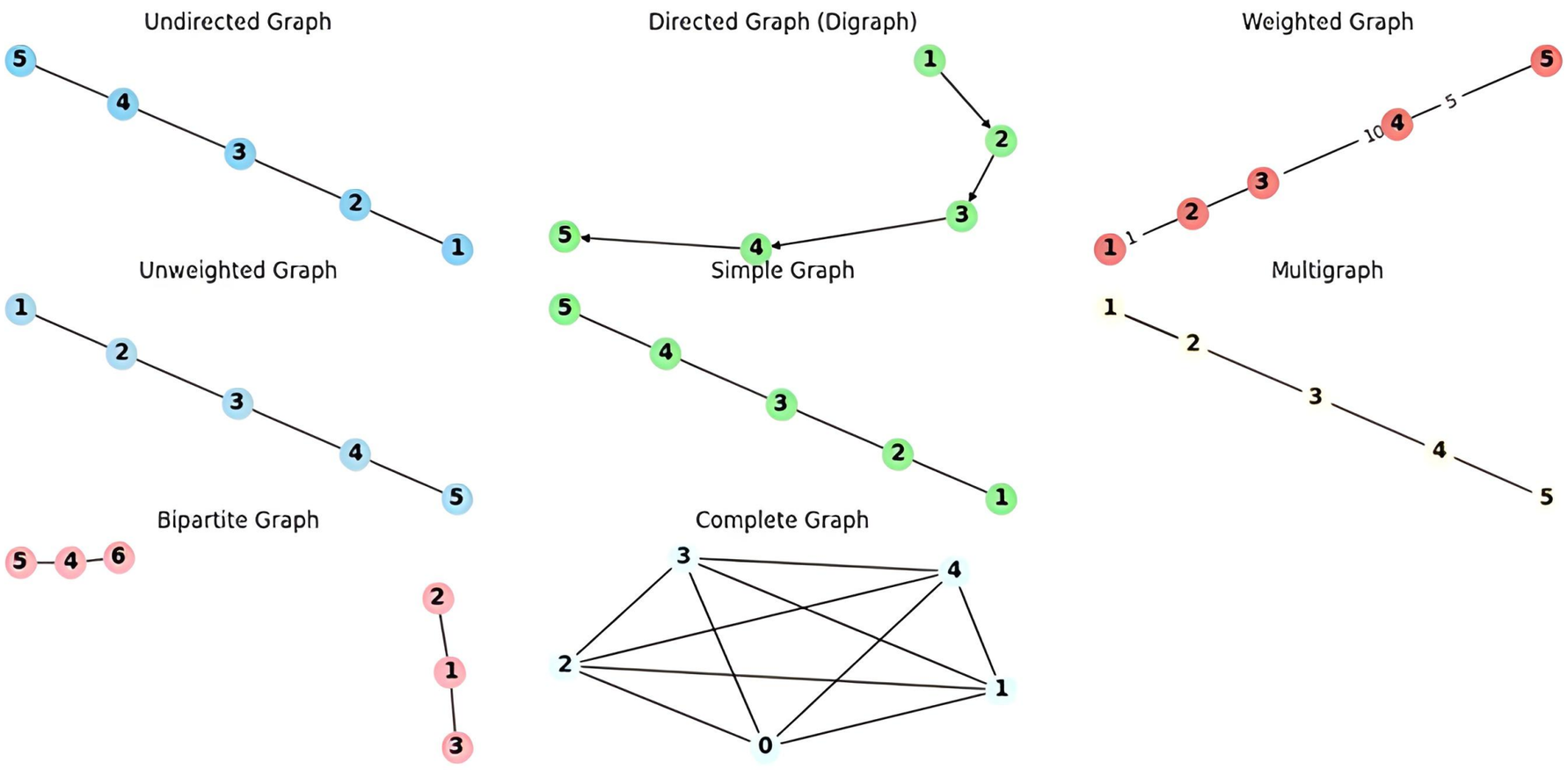
Graph

AI / ML powered apps and wider, deeper analytics



Fraud detection, compliance, supply chain, customer 360, predictive maintenance, IoT Analysis, and EMS

Types of Graphs



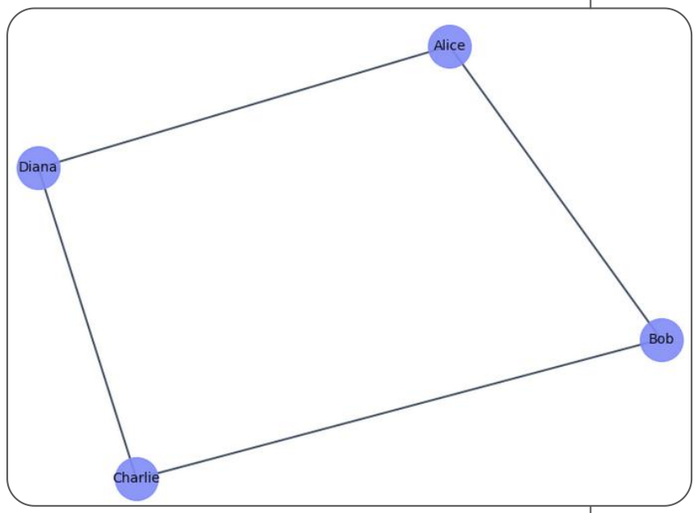
Types of Graphs

- Using python library ***NetworkX***

Type	NetworkX class	Key feature	Example
Graph	`nx.Graph()`	Undirected, simple	Friendship network
DiGraph	`nx.DiGraph()`	Directed edges A→B ≠ B→A	Twitter follows, web links
MultiGraph	`nx.MultiGraph()`	Multiple edges between same pair	Flight routes (airlines)
MultiDiGraph	`nx.MultiDiGraph()`	Directed + multiple edges	Transaction flows
Weighted Graph	`any + weight`	Edges carry numeric strength	Road distances, correlation
Bipartite Graph	`nx.Graph() + annotation`	Two disjoint node sets	Users ↔ Products

Building Your First Graph

```
import networkx as nx
# Create graph
G = nx.Graph()
# Add nodes individually or in bulk
G.add_node("Alice", age=30, role="admin")
G.add_nodes_from(["Bob", "Charlie", "Diana"])

# Add edges individually or in bulk
G.add_edge("Alice", "Bob", weight=0.9)
G.add_edges_from([
    ("Bob", "Charlie", {"weight": 0.5}),
    ("Charlie", "Diana"),
    ("Diana", "Alice")
])

print(G.number_of_nodes(), G.number_of_edges())
```



Output: 4 nodes, 4 edges

ACCESSING DATA

G.nodes["Alice"]["age"] -> 30

G["Alice"]["Bob"]["weight"] -> 0.9

DIRECTED GRAPH

Replace nx.Graph() with nx.DiGraph().

All other syntax stays the same.

FROM PANDAS

nx.from_pandas_edgelist(df, "src", "dst") builds a graph from a DataFrame in one line.

Key Graph Metrics

- Used to quantify the structural properties of networks, allowing us to move from visual inspection to mathematical analysis.
- Help to identify key influencers, bottlenecks, community structures, and the overall robustness of complex systems in fields like social science, infrastructure, and biology.

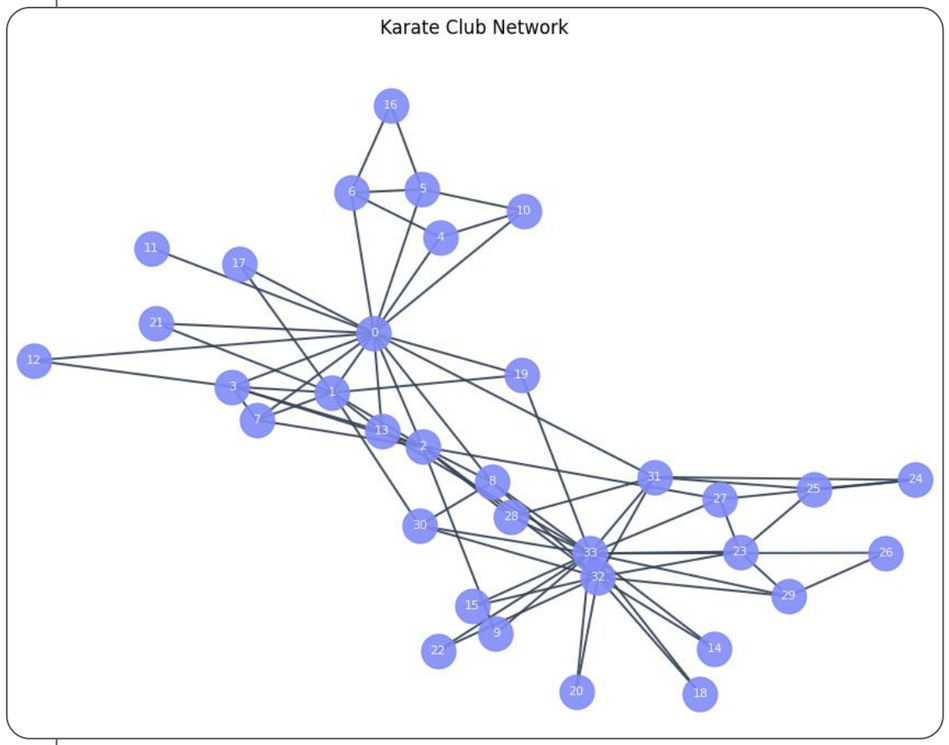
- **Degree**
  - How many direct connections a node has. High-degree nodes are "hubs".
- **Betweenness centrality**
  - How often a node lies on the shortest path between others. Reveals bridge nodes.
- **Clustering coefficient**
  - How tightly a node's neighbours are connected to each other (0 = none, 1 = clique).
- **Density**
  - Ratio of existing edges to maximum possible edges.
  - Sparse = 0.0, dense = 1.0.

```
import networkx as nx
G = nx.karate_club_graph()

# Structure
nx.is_connected(G)           # True/False
nx.density(G)                # 0.0–1.0
nx.diameter(G)               # longest short path
nx.average_clustering(G)     # clique tendency

# Node importance
nx.degree_centrality(G)      # connection count
nx.betweenness_centrality(G) # bridge nodes
nx.pagerank(G)               # influence score

# Components
nx.number_connected_components(G)
list(nx.connected_components(G))
```

Graph Visualization

- Drawing with Matplotlib

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()

# Choose layout algorithm
pos = nx.spring_layout(G, seed=42)

plt.figure(figsize=(10, 7))

nx.draw_networkx(G, pos,
    node_color="#818cf8",
    node_size=500,
    edge_color="#334155",
    font_color="white",
    font_size=8,
    width=1.5,
    alpha=0.9
)
plt.title("Karate Club Network")
plt.axis("off")
plt.tight_layout()
plt.show()
```



Key parameters:

NODE_COLOR

Hex string, list of values (for colormap), or list of colors per node.

NODE_SIZE

Integer or list. Scale by degree: [G.degree(n)*30 for n in G]

WITH_LABELS

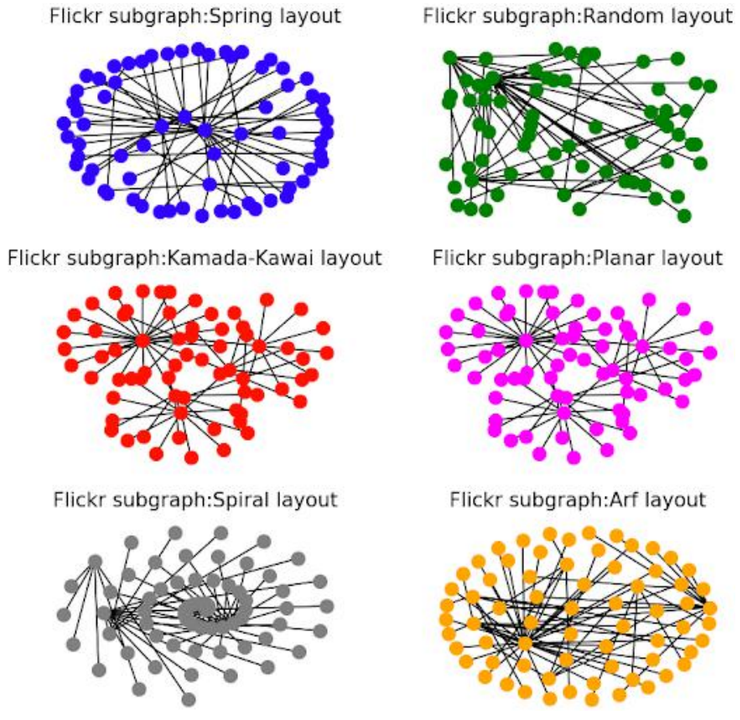
Set False for dense graphs. Use draw_networkx_labels() separately for fine control.

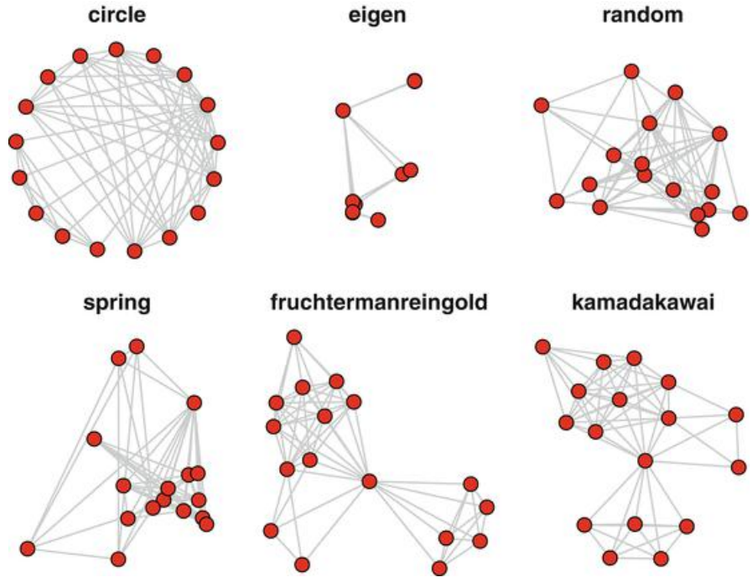
Layout Algorithms

- The layout determines node positions, different algorithms reveal different structural patterns.

Layout	Function	Best for	Speed
Spring	`spring_layout(G, seed=42)`	General purpose, force-directed	Medium
Circular	`circular_layout(G)`	Evenly spaced, small graphs	Fast
Kamada-Kawai	`kamada_kawai_layout(G)`	Aesthetically pleasing	Slow
Shell	`shell_layout(G, nlist=groups)`	Concentric circles by group	Fast
Spectral	`spectral_layout(G)`	Reveals clusters via eigenvectors	Medium
Random	`random_layout(G, seed=0)`	Baseline / debugging only	Fastest

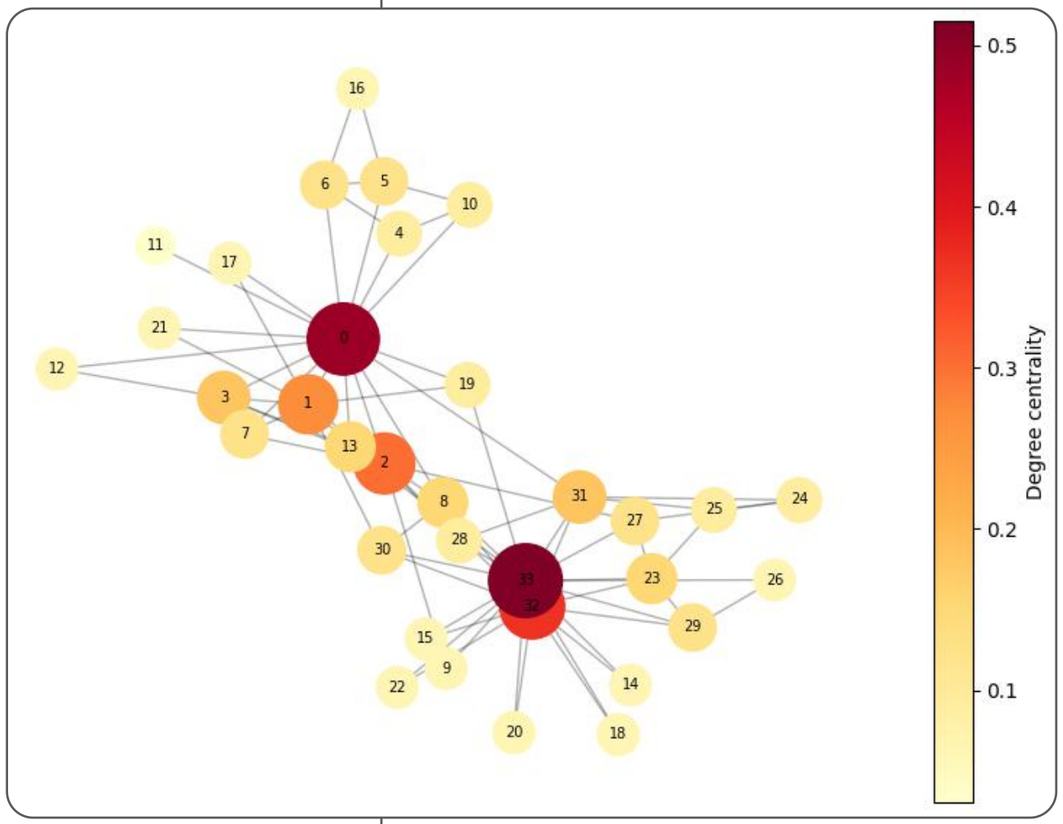
- Always pass `seed=42` to `spring_layout`, without it the layout changes every run, making reproducibility impossible.





Color Nodes by Attribute

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.karate_club_graph()
pos = nx.spring_layout(G, seed=42)

# Color by degree centrality
centrality = nx.degree_centrality(G)
colors = [centrality[n] for n in G.nodes]
sizes  = [300 + centrality[n]*2000 for n in G.nodes]

fig, ax = plt.subplots(figsize=(10, 7))
sc = nx.draw_networkx_nodes(G, pos,
    node_color=colors,
    cmap=plt.cm.YlOrRd,
    node_size=sizes, ax=ax)
nx.draw_networkx_edges(G, pos, alpha=0.3, ax=ax)
nx.draw_networkx_labels(G, pos, font_size=7, ax=ax)

plt.colorbar(sc, ax=ax, label="Degree centrality")
plt.axis("off")
plt.show()
```



WHAT TO ENCODE WITH COLOR

Centrality - spot hubs

Community - reveal clusters

Node type - categorical

Anomaly score - outliers

GOOD COLORMAPS

YlOrRd - sequential importance

RdYlGn - good vs bad

tab10 - 10 distinct categories

viridis - perceptually uniform

NetworkX + Matplotlib works well up to ~400 nodes. For bigger graphs, use specialized tools pyvis, plotly, graph with C++ backend.

Why Graph Cleaning Matters

Dirty graphs produce silently wrong results.

Unlike a NaN in a DataFrame, a bad edge does not throw an error, it just corrupts your metrics.

1. **Self-loops** inflate degree counts and break clustering calculations.
2. **Duplicate edges** make a simple graph behave like a multigraph - wrong centrality values.
3. **Isolated nodes** break is_connected() and skew average-degree metrics.
4. **Noise edges** with tiny weights pollute shortest-path and community detection.
5. **Missing attributes** cause KeyError crashes mid-pipeline.

CLEANING ORDER MATTERS

1. Remove self-loops
2. Deduplicate edges
3. Filter by weight
4. Remove isolated nodes
5. Normalize attributes

Diagnose First

Always run diagnose_graph() before touching the graph. It tells you exactly which cleaning steps are needed.

```
import networkx as nx

def diagnose_graph(G):
    print("=== Graph Report ===")
    print("Nodes         : " + str(G.number_of_nodes()))
    print("Edges         : " + str(G.number_of_edges()))
    print("Self-loops    : " + str(nx.number_of_selfloops(G)))
    print("Isolated nodes: " + str(len(list(nx.isolates(G)))))
    print("Components    : " + str(nx.number_connected_components(G)))
    print("Is connected  : " + str(nx.is_connected(G)))
    print("Density       : " + str(round(nx.density(G), 4)))

    # Check attribute coverage
    attrs = ["label", "weight", "type"]
    for attr in attrs:
        missing = sum(1 for n in G.nodes
                      if attr not in G.nodes[n])
        print("Missing '" + attr + "': " + str(missing) + " nodes")

diagnose_graph(G)
```

WHAT TO LOOK FOR

Self-loops > 0
Isolated nodes > 0
Components > 1
Missing attributes

Output:

```
=== Graph Report ===
Nodes         : 34
Edges         : 78
Self-loops    : 3
Isolated nodes: 2
Components    : 4
Is connected  : False
Missing label : 5 nodes
```

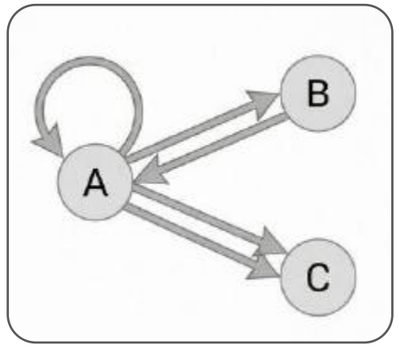
Remove Self-loops & Duplicates

Self-loops (A -> A)

```
# Detect
loops = list(nx.selfloop_edges(G))
print("Found " + str(len(loops)) + " self-loops")

# Remove all self-loops in one call
G.remove_edges_from(
    nx.selfloop_edges(G)
)

# Verify
assert nx.number_of_selfloops(G) == 0
```

Output:

Found 3 self-loops: [(A,A), (C,C), (F,F)]



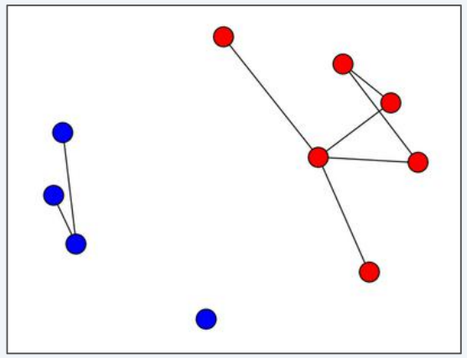
Duplicate edges (A -> C, A -> C)

```
# Method 1: convert MultiGraph to Graph
# Automatically keeps one edge per pair
G_simple = nx.Graph(G)

# Method 2: detect manually
seen = set()
dupes = []
for u, v in G.edges():
    key = tuple(sorted([u, v]))
    if key in seen:
        dupes.append((u, v))
    else:
        seen.add(key)

print("Removing " + str(len(dupes)) + " duplicates")
G.remove_edges_from(dupes)
```

Remove Isolated Nodes & Weak Edges

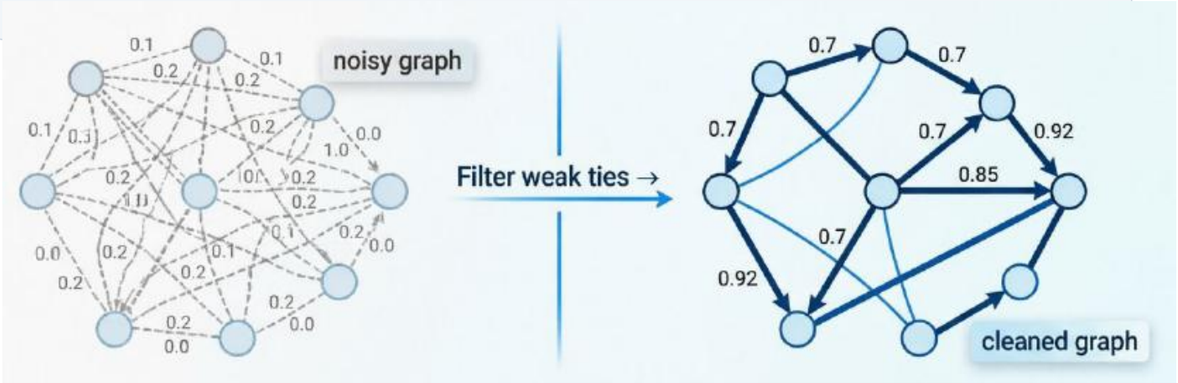
Isolated nodes (degree = 0)

```
# List isolated (zero-connection) nodes
isolated = list(nx.isolates(G))
print("Isolated: " + str(isolated))

# Remove them
G.remove_nodes_from(isolated)

# Alternative: tag instead of delete
for node in nx.isolates(G):
    G.nodes[node]["isolated"] = True

# Low-degree pruning (degree < 2)
low = [n for n, d in G.degree() if d < 2]
G.remove_nodes_from(low)
```



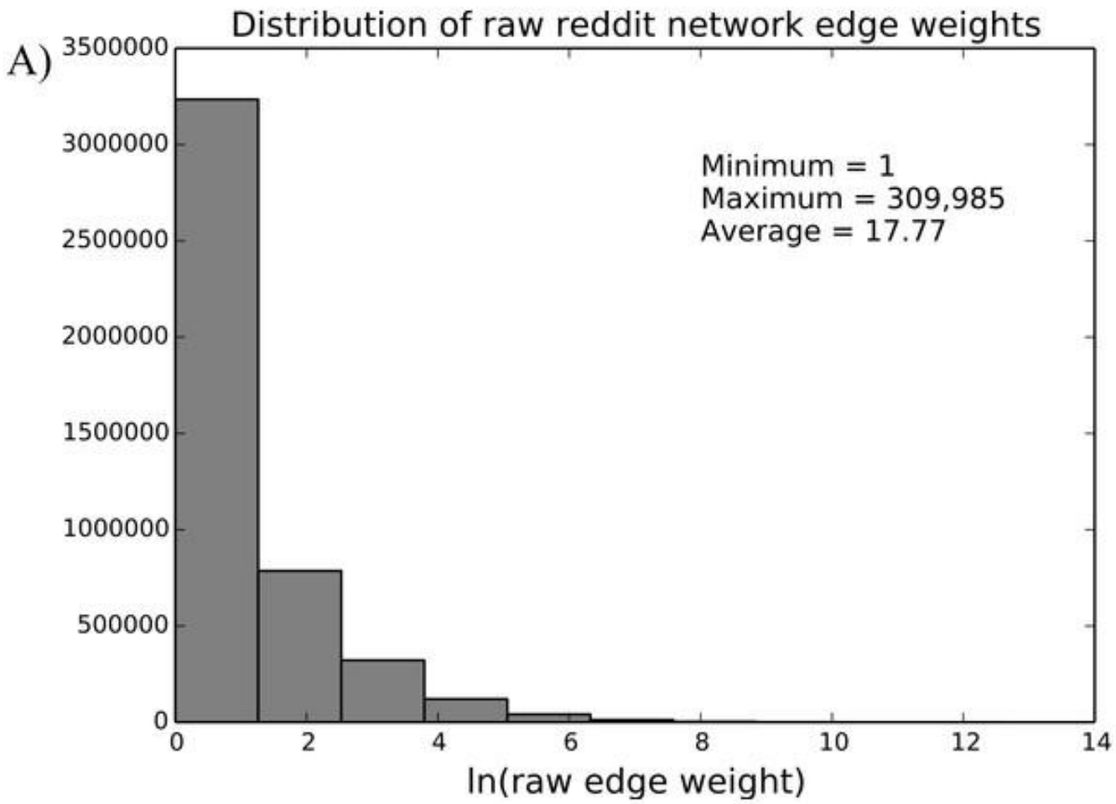
Weak edges (filter by weight)

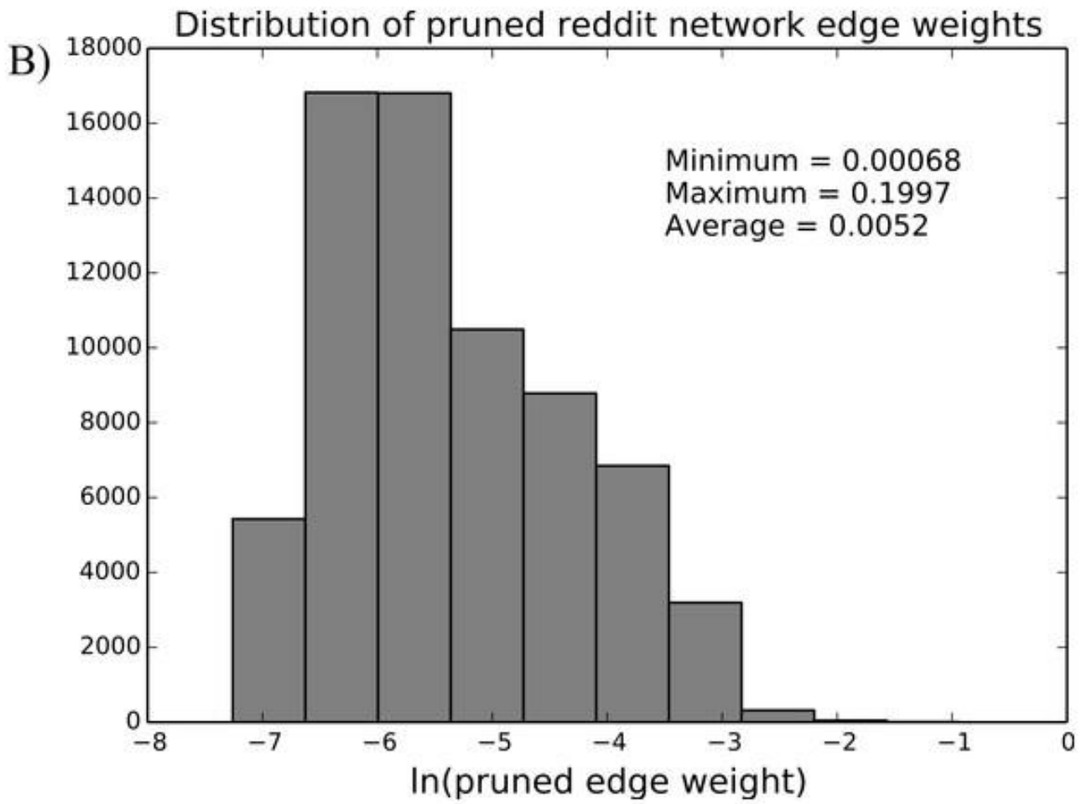
```
# Inspect weight distribution first
weights = [d["weight"]
           for _,_,d in G.edges(data=True)
           if "weight" in d]
print("Min: " + str(round(min(weights), 2)))
print("Max: " + str(round(max(weights), 2)))

# Remove edges below threshold
threshold = 0.2
weak = [(u, v) for u, v, d in
        G.edges(data=True)
        if d.get("weight", 1) < threshold]
G.remove_edges_from(weak)
```



Improve readability and informativness.

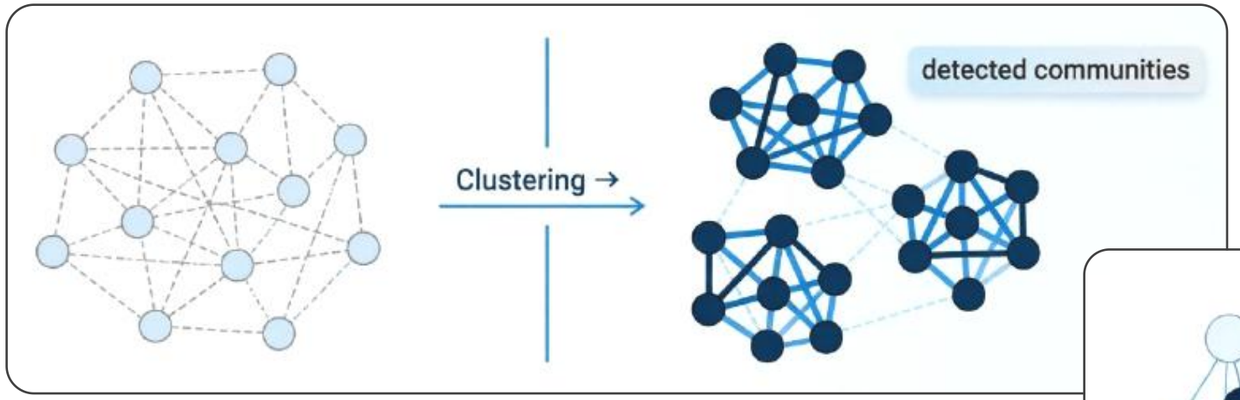
Removing edges and impact on disribution of weights

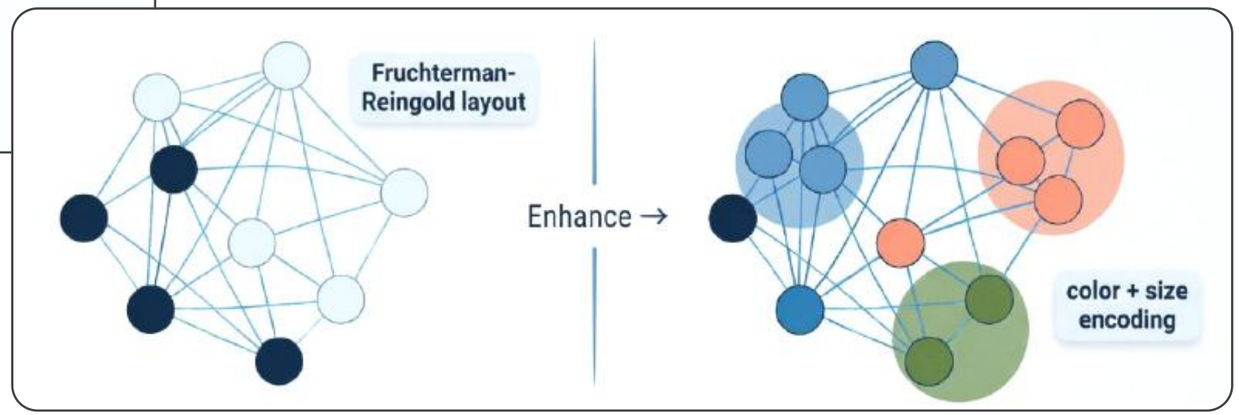




Comunity Detection

- Community detection uncovers hidden group structures and enables ***interpretable*** network ***segmentation***.
  - ***Identify clusters*** of densely connected nodes
  - ***Modularity and conductance metrics*** evaluate community quality
  - Algorithms detect ***latent communities*** present in ***textual or relational data***





Normalize & Fill Missing Attributes

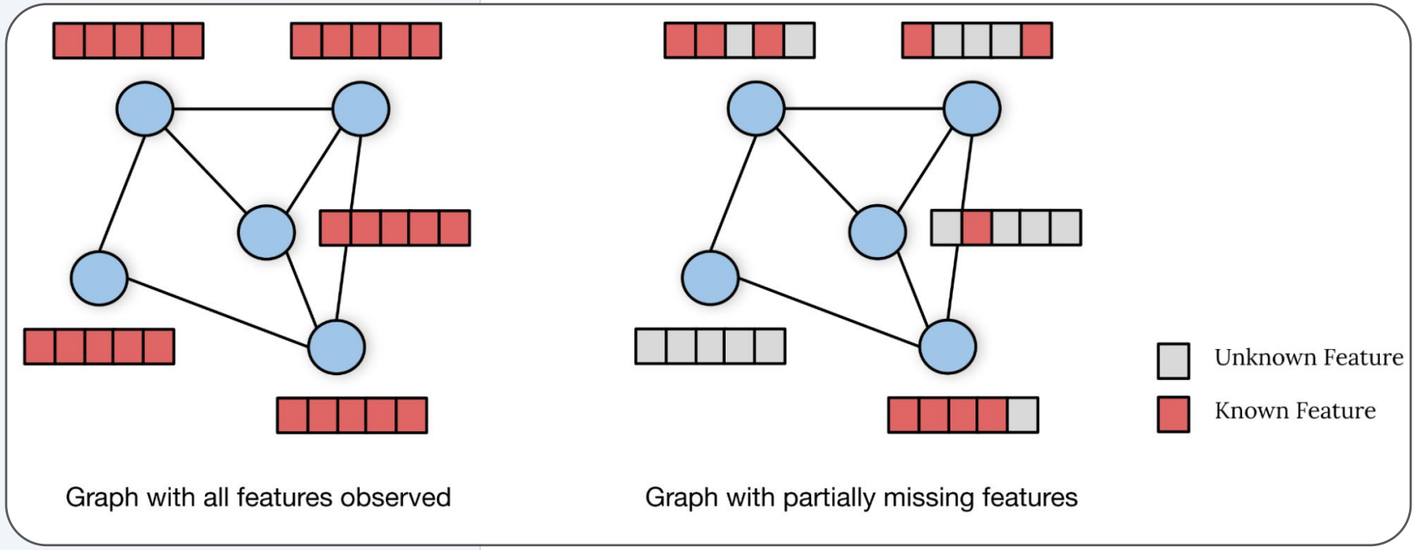
```
# Fill missing node attributes
for node in G.nodes:
    attrs = G.nodes[node]
    # Set defaults for missing keys
    attrs.setdefault("label", str(node))
    attrs.setdefault("weight", 1.0)
    attrs.setdefault("type", "unknown")
    # Normalize string labels
    attrs["label"] = attrs["label"].strip().lower()

# Fill missing edge attributes
for u, v, data in G.edges(data=True):
    data.setdefault("weight", 1.0)
    data.setdefault("relation", "unknown")
# Remove nodes with null/empty names
null_nodes = [
    n for n in G.nodes
    if not G.nodes[n].get("label")
]
G.remove_nodes_from(null_nodes)
print("Removed " + str(len(null_nodes)) + " unnamed nodes")
```

WHY NORMALIZE LABELS?

"Alice", " alice ", "ALICE" are three representations of the same node. Without normalization, they become three separate nodes.

TYPE COERCION

Ensure numeric attributes are actually numeric: G.nodes[n]["age"] = int(attrs.get("age", 0))



Filling missing attributes maintains the graph's structural integrity, allowing algorithms to function without breaking the connections between nodes.

Lab